1 Mod and AddMod

// compute x%y
// x, y are n-bit integer, in big endian
// $\{x_{n-1}, x_{n-2}, \ldots, x_0\}$ is binary form of x
// $\{y_{n-1}, y_{n-2}, \ldots, y_0\}$ is binary form of y
// guarantee intermediate variables are in n-bit.

MOD(x, y):

```
// rem is n-bit integer
rem = 0
for i = 0 to n − 1:
    rem = rem << 1
    rem = rem + x_i
    if rem >= y:
        rem = rem − y
return rem
```

// compute (x + y)%p
// x, y, p are n-bit integer
// guarantee intermediate variables are in n-bit

AddMod(x, y, p):

```
x = x % p // x < p
y = y % p // y < p
if x ⩾ p − y:
    return x − (p − y)
return x + y
```

2 MulMod and ExpMod

// compute (xy)%p
// x, y, p are n-bit integer, in big-endian
// $\{x_{n-1}, x_{n-2}, \ldots, x_0\}$ is binary form of x
// $\{y_{n-1}, y_{n-2}, \ldots, y_0\}$ is binary form of y
// We have $y = \sum_{i=0}^{n-1} y_i \cdot 2^i$.
// Hence, $xy = x \sum_{i=0}^{n-1} y_i \cdot 2^i = \sum_{i=0}^{n-1} x \cdot y_i \cdot 2^i$.
// guarantee intermediate variables are in n-bit

MultMod(x, y, p):

```
ret = 0
x = x%p
for i = 0 to n − 1:
    if y_i == 1:
        ret = AddMod(ret, x, p) // ret = (ret + x)%p
    x = AddMod(x, x, p) // x = (2x)%p
return ret
```

// compute (x^y)%p
// x, y, p are n-bit integer, in big-endian
// $\{x_{n-1}, x_{n-2}, \ldots, x_0\}$ is binary form of x
// $\{y_{n-1}, y_{n-2}, \ldots, y_0\}$ is binary form of y
// We have $y = \sum_{i=0}^{n-1} y_i \cdot 2^i$.
// Hence, $x^y = x^{\sum_{i=0}^{n-1} y_i \cdot 2^i}$.
// guarantee intermediate variables are in n-bit

ExpMod(x, y, p):

```
ret = 1
base = x%p
for i = 0 to n − 1:
    if y_i == 1:
        ret = MultMod(ret, base, p) // ret = ret * base
    base = MultMod(base, base, p) // base = base * base
return ret
```